NetTower – Sprint 1 Review

Benjamin Molloy
ASE Capstone
March 2, 2026

Sprint 1: Backend Discovery & Modeling Complete

Problem Reminder

Small or disrupted networks lack simple situational awareness tools.

- Enterprise tools:
  - Too heavy
  - Too complex
  - Infrastructure dependent

NetTower Goal: Provide lightweight, deployable network topology awareness.

Sprint 1: Scope & Completion

Sprint 1 Focus: Backend Discovery & Relationship Modeling

Planned Scope

- Establish project architecture
- Implement agentless discovery
- Model hosts and inferred relationships
- Stabilize backend pipeline
- Complete working prototype of back end

Milestones & Deadlines Met

- Milestone 1 – February 16, 2026
  - Planning & Backend Architecture Finalized
- Hard Deadline 1 – March 9, 2026
  - Backend Core Complete (Sprint 1 Presentation)

Completed Work

- Core backend architecture established
- Event models & EventBus system implemented
- MongoDB integration complete
- Passive + active discovery integrated
- Host correlation logic implemented
- Connectivity (edge) modeling functional
- Enrichment (OUI + device hints) added
- Orchestrator loop stabilized

Sprint 1: Burn Down

<u>Burn Down</u>

- Total Defined Requirements: **16**

- **Backend Sprint 1 Requirements 4/4**
  - 100% Complete

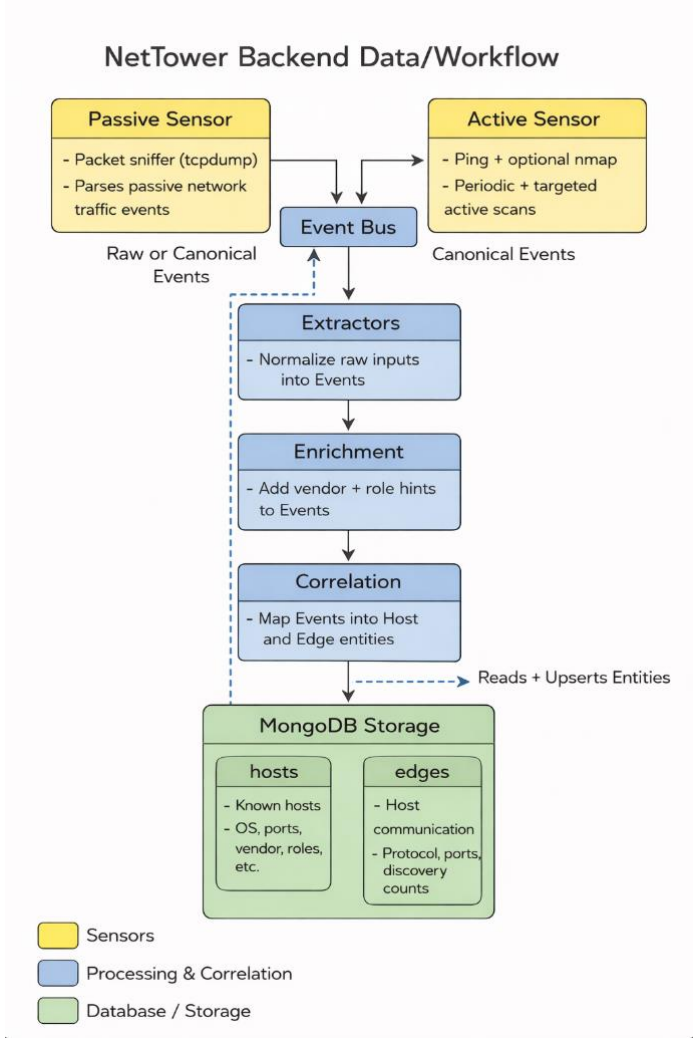
- **Total Requirements: 4 / 16**
  - ~25% Complete

- Remaining Requirements: **12**
  - Remaining Focus: Visualization, interaction, UI integration (Sprint 2 focus)

Total Lines of Code: 2,986

[ ] 4 / 16

Sprint 1: Demo Part 1



Code Structure:

```
NetTower/
└─code/
  ├─ frontEnd/
  │  ├─ app.py
  │  ├─ templates/
  │  │  └─ index.html
  │  └─ static/
  │     ├─ css/
  │     └─ js/
  │
  └─ backEnd/
     ├─ __init__.py
     ├─ main.py
     ├─ config.py
     ├─ settings.py
     │
     ├─ models/
     │  ├─ __init__.py
     │  ├─ events.py
     │  ├─ entities.py
     │  └─ types.py
     │
     ├─ pipeline/
     │  └─ event_bus.py
     │
     ├─ sensors/
     │  ├─ passive_listener.py
     │  └─ active_discovery.py
     │
     ├─ processors/
     │  ├─ extractors.py
     │  ├─ enrichment.py
     │  └─ correlation.py
     │
     ├─ storage/
     │  ├─ mongo_client.py
     │  └─ librarian.py
     │
     ├─ resources/
     │  ├─ oui_vendors.json
     │  └─ device_hints.json
     │
     └─ utils/
        ├─ logging.py
        └─ net.py
```

Sprint 1: Demo Part 2

Logging:

```
2026-03-02 16:41:55 | INFO | backEnd.main | NetTower backend starting...
2026-03-02 16:41:55 | INFO | backEnd.main | Passive listener enabled.
2026-03-02 16:41:55 | INFO | backEnd.sensors.passive_listener | passive listener started on iface=en0 filter='arp or ip'

^C2026-03-02 16:50:24 | INFO | backEnd.main | KeyboardInterrupt received, shutting down...
2026-03-02 16:50:24 | INFO | backEnd.sensors.passive_listener | passive listener stopped
2026-03-02 16:50:24 | INFO | backEnd.main | NetTower backend stopped.
```

Collected Hosts:

```
benmolloy@nku23R0171:~/Documents/NetTower-Capstone-Project/code^main ±
% venv/bin/python -c "from pymongo import MongoClient; db=MongoClient('mongodb://localhost:27017')['nettower']; [print(h) for h in db.hosts.find()]"
{'_id': ObjectId('69a6014de01164d17f4aaa32'), 'host_id': 'host_ip_10_18_212_244', 'first_seen': datetime.datetime(2026, 3, 2, 21, 29, 49, 885000), 'ips': ['10.18.2
12.244'], 'last_seen': datetime.datetime(2026, 3, 2, 21, 47, 1, 22000), 'macs': [], 'os_guess': None, 'ports': [], 'vendor': None}
{'_id': ObjectId('69a6014de01164d17f4aaa33'), 'host_id': 'host_ip_172_28_116_111', 'first_seen': datetime.datetime(2026, 3, 2, 21, 29, 49, 885000), 'ips': ['172.28
.116.111'], 'last_seen': datetime.datetime(2026, 3, 2, 21, 46, 58, 794000), 'macs': [], 'os_guess': None, 'ports': [], 'vendor': None}
{'_id': ObjectId('69a6014ee01164d17f4aaa35'), 'host_id': 'host_ip_10_11_0_51', 'first_seen': datetime.datetime(2026, 3, 2, 21, 29, 49, 986000), 'ips': ['10.11.0.51
'], 'last_seen': datetime.datetime(2026, 3, 2, 21, 47, 1, 22000), 'macs': [], 'os_guess': None, 'ports': [], 'vendor': None}
{'_id': ObjectId('69a6014ee01164d17f4aaa37'), 'host_id': 'host_ip_10_18_208_241', 'first_seen': datetime.datetime(2026, 3, 2, 21, 29, 50, 189000), 'ips': ['10.18.2
08.241'], 'last_seen': datetime.datetime(2026, 3, 2, 21, 29, 53, 128000), 'macs': [], 'os_guess': None, 'ports': [], 'vendor': None}
```

Collected Edges:

```
{'_id': ObjectId('69a6048ae01164d17f4aab1b'), 'edge_key': 'host_ip_10_18_208_49__host_ip_10_18_212_244__TCP', 'a_host_id': 'host_ip_10_18_208_49', 'b_host_id': 'ho
st_ip_10_18_212_244', 'count': 10, 'first_seen': datetime.datetime(2026, 3, 2, 21, 43, 38, 663000), 'last_seen': datetime.datetime(2026, 3, 2, 21, 47, 33, 742000),
 'ports': [[5353, 5353]], 'proto': 'TCP'}
{'_id': ObjectId('69a6048ee01164d17f4aab1d'), 'edge_key': 'host_ip_10_18_208_47__host_ip_10_18_212_244__TCP', 'a_host_id': 'host_ip_10_18_208_47', 'b_host_id': 'ho
st_ip_10_18_212_244', 'count': 7, 'first_seen': datetime.datetime(2026, 3, 2, 21, 43, 42, 13000), 'last_seen': datetime.datetime(2026, 3, 2, 21, 47, 33, 32000), 'p
orts': [[5353, 5353]], 'proto': 'TCP'}
{'_id': ObjectId('69a6048ee01164d17f4aab1e'), 'edge_key': 'host_ip_10_18_208_47__host_ip_224_0_0_251__TCP', 'a_host_id': 'host_ip_10_18_208_47', 'b_host_id': 'host
_ip_224_0_0_251', 'count': 15, 'first_seen': datetime.datetime(2026, 3, 2, 21, 43, 42, 13000), 'last_seen': datetime.datetime(2026, 3, 2, 21, 48, 13, 79000), 'port
s': [[5353, 5353]], 'proto': 'TCP'}
{'_id': ObjectId('69a6048fe01164d17f4aab20'), 'edge_key': 'host_ip_104_208_16_90__host_ip_10_18_212_244__TCP', 'a_host_id': 'host_ip_104_208_16_90', 'b_host_id': '
host_ip_10_18_212_244', 'count': 37, 'first_seen': datetime.datetime(2026, 3, 2, 21, 43, 43, 231000), 'last_seen': datetime.datetime(2026, 3, 2, 21, 45, 43, 421000
), 'ports': [[443, 61425], [61425, 443]], 'proto': 'TCP'}
{'_id': ObjectId('69a60492e01164d17f4aab22'), 'edge_key': 'host_ip_10_18_208_112__host_ip_224_0_0_251__TCP', 'a_host_id': 'host_ip_10_18_208_112', 'b_host_id': 'ho
st_ip_224_0_0_251', 'count': 10, 'first_seen': datetime.datetime(2026, 3, 2, 21, 43, 46, 781000), 'last_seen': datetime.datetime(2026, 3, 2, 21, 49, 26, 747000), '
ports': [[5353, 5353]], 'proto': 'TCP'}
{'_id': ObjectId('69a60492e01164d17f4aab24'), 'edge_key': 'host_ip_10_18_212_244__host_ip_52_96_163_18__TCP', 'a_host_id': 'host_ip_10_18_212_244', 'b_host_id': 'h
ost_ip_52_96_163_18', 'count': 249, 'first_seen': datetime.datetime(2026, 3, 2, 21, 43, 46, 882000), 'last_seen': datetime.datetime(2026, 3, 2, 21, 49, 28, 872000)
, 'ports': [[443, 51385], [51385, 443]], 'proto': 'TCP'}
```

```
% venv/bin/python -c "from p
('edges', db.edges.count_doc
hosts 141
edges 182
```

Learning with AI - Packet Structure & Protocol Breakdown

Objective: Understand packet behavior from frame header to payload.

Studied:

- Ethernet, IP, TCP, UDP, ICMP headers
- Flags and response codes
- What indicates reachability vs filtering

AI Used For:

- Breaking down protocol mechanics step-by-step
- Explaining open vs closed vs filtered behavior
- Clarifying what packet signals truly indicate

Impact on NetTower:

- Improved interpretation of probe results
- Reduced incorrect assumptions about reachability

Learning with AI - L2, L3, L4 Behavior, Constraints & Design Integration

Objective: Understand what Layers 2–4 expose — and what they do not.

Studied:

- Effects of NAT, firewalls, segmentation
- Timeouts vs rejects
- Visibility limits in constrained networks

AI Used For:

- Exploring misleading signals
- Stress-testing inference assumptions
- Validating cautious reasoning

Impact on NetTower:

- Clearer distinction between confirmed vs inferred conclusions
- More accurate uncertainty representation in visualization design
- Stronger architectural decisions grounded in protocol behavior
- AI supports technical learning — not product features.

Risks Identified

- Incomplete or misleading network data
- Overconfidence in inferred relationships
- Segmented or filtered environments
- Time constraints

<u>Mitigation:</u>

- Explicit uncertainty modeling
- Incremental architectural validation

Sprint 2 Plan

- Sprint 2 Focus: Frontend, Visualization Documentation and Testing
  - Build topology interface
  - Integrate backend API
  - Implement interactive navigation
  - Add heat mapping & visual differentiation

- Goal: Functional prototype (Backend + Visualization)

<u>Sprint 2 Schedule:</u>

- S2W1 (Week 10 – 3/16) Building Front End
- S2W2 (Week 11 – 3/23) Refinement of front end and Visual enhancement
- S2W3 (Week 12 – 3/30) Frontend–Backend Integration & Topology Rendering
- S2W4 (Week 13 – 4/6) Performance Optimization & System Refinement + User Testing
- S2W5 (Week 14 – 4/13) Documentation and publication prep
- S2W6 (Week 15 – 4/20) Final testing, demo, and submission

Closing Summary

<u>Sprint 1 Outcome:</u>

- Backend discovery pipeline operational
- Host and relationship modeling functional
- Architecture stabilized
- Technical assumptions validated through AI-supported learning

Next: NetTower is ready for visualization development in Sprint 2.

Questions?